

◇ useRef Hook in React – Short Explanation

The **useRef** hook is used to create a **mutable reference** that persists across renders **without causing a re-render**.

☑ When to Use useRef?

1. Accessing DOM elements (like **input**, **button**)
 2. Storing mutable values without re-rendering
 3. Keeping a reference to previous state values
-

🔗 1. Accessing DOM Elements

👉 Use **useRef** to focus an input field:

```
import { useRef } from "react";

function FocusInput() {
  const inputRef = useRef(null);

  const handleFocus = () => {
    inputRef.current.focus(); // Directly interacts with the DOM
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Type here..." />
      <button onClick={handleFocus}>Focus Input</button>
    </div>
  );
}

export default FocusInput;
```

☑ Why?

- **useRef** stores a reference to the input field without re-rendering.
 - **inputRef.current.focus()** directly focuses the input field.
-

🔗 2. Storing Mutable Values Without Re-Rendering

👉 Use **useRef** to store a variable without triggering re-renders:

```
import { useRef, useState } from "react";

function Counter() {
```

```

const countRef = useRef(0);
const [stateCount, setStateCount] = useState(0);

const incrementRef = () => {
  countRef.current += 1; // Changes value but does NOT re-render
  console.log("Ref Count:", countRef.current);
};

return (
  <div>
    <p>State Count: {stateCount}</p>
    <button onClick={() => setStateCount(stateCount + 1)}>Increment
State</button>
    <button onClick={incrementRef}>Increment Ref</button>
  </div>
);
}

export default Counter;

```

☑ Why?

- `useRef` updates `countRef.current` **without causing a re-render**.
- **Unlike `useState`**, `useRef` values persist between renders without updating the UI.

🔑 3. Keeping Previous State Values

👉 Use `useRef` to remember the previous state value:

```

import { useRef, useEffect, useState } from "react";

function PreviousStateExample() {
  const [count, setCount] = useState(0);
  const prevCountRef = useRef(null);

  useEffect(() => {
    prevCountRef.current = count; // Store previous count value
  }, [count]);

  return (
    <div>
      <p>Current Count: {count}</p>
      <p>Previous Count: {prevCountRef.current}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

export default PreviousStateExample;

```

☑ Why?

- `useRef` stores the **previous count** without causing re-renders.
 - Every time `count` updates, `prevCountRef.current` keeps the last value.
-

🔑 Key Takeaways

- **Does NOT** cause re-renders like `useState`.
- Can access and modify DOM elements (`ref={inputRef}`).
- Useful for storing previous state values (`prevCountRef.current`).

🚀 Use `useRef` when you need to store values across renders but don't want to trigger a re-render!